SQL to ORM Refactoring and Security Analysis

Python | SQLAlchemy | MySQL

This assignment examines the use of an AI assistant to refactor procedural database code into a SQLAlchemy ORM solution and evaluate its security, abstraction, and maintainability benefits.

Prepared by: Prince Ubakaeze

Submission date: 04/10/2026

Prompt Formulation
Execution Screenshot
Verification and Reflection
GitHub Repository Update

Prompt Formulation

I have a procedural Python program that uses `mysql.connector`, cursors, and raw SQL to perform CRUD operations on a MySQL `users` table. I want you to refactor it into a modern, secure, professional, object-oriented implementation using the SQLAlchemy Object-Relational Mapper (ORM).

The existing program connects to a MySQL database and performs operations such as creating a user, finding a user by username, updating a user's email, deleting a user, and listing all users.

Please refactor the application with the following requirements:

Use SQLAlchemy ORM rather than manually executing raw SQL statements. Define a `User` class as a SQLAlchemy declarative model mapped to a `users` table. Give the `User` model appropriate fields including an integer primary key, username, and email. Make username and email required, and use appropriate constraints such as uniqueness where reasonable.

Configure a SQLAlchemy engine for a MySQL database and create an ORM `Session`on. Show explicitly how SQLAlchemy creates the database table using the model metadata.

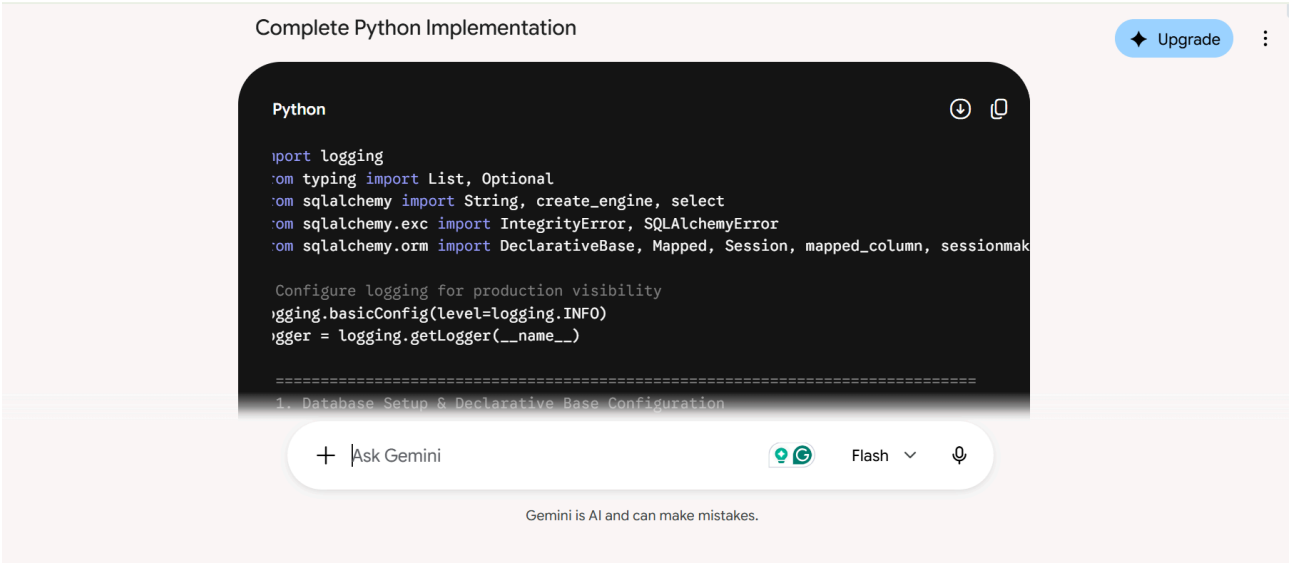
Implement professional ORM versions of the following operations: create a user; retrieve a user by username; update a user's email; delete a user; list all users. Include basic input validation and appropriate exception handling. Use proper transaction management, including committing successful changes and rolling back failed transactions.

Demonstrate the code by creating the tables, adding a user, and querying for that user using an ORM Session. Do not perform the CRUD operations by formatting or concatenating raw SQL strings.

After the code, explain in detail why the SQLAlchemy ORM approach is more secure and professional than a procedural raw-SQL implementation, particularly one that uses SQL string formatting. In the explanation, discuss SQL injection risk, parameter binding, abstraction, maintainability, code duplication, schema representation, transaction management, readability, testability, portability, and long-term development. Also explain that using an ORM does not automatically make every database operation secure if a developer later introduces unsafe raw SQL.

Please provide complete, executable Python code followed by a detailed technical explanation.

Execution Screenshot



Verification and Reflection

Using an ORM changes database programming from thinking primarily in terms of SQL statements to thinking in terms of application objects. With SQLAlchemy, a `User` object represents a row in the `users` table, and the model class defines the structure and constraints of that data in one central place. I find this approach less error-prone because I do not have to repeatedly remember table names, column names, SQL syntax, placeholders, or the correct order of values every time I perform a database operation. The ORM also reduces the number of manually constructed SQL strings in the program, which lowers the chance of mistakes involving quoting, escaping, incorrect parameters, or unsafe incorporation of user input.

From a maintainability perspective, the object-centered design becomes more valuable as an application grows. If the database structure or business requirements change, developers can often update the model and related application methods instead of searching through many separate SQL statements. SQLAlchemy sessions also provide a consistent way to query data, manage transactions, commit successful changes, and roll back failed operations. This makes the program easier to test, extend, and understand when several developers work on the same project. The ORM therefore provides more than shorter syntax: it creates a structured layer between the application and the database that makes long-term development more predictable and manageable.

GitHub Repository Update